A Practical Guide to Exploratory Data Analysis

An 11-step, reusable workflow you can apply to any dataset

11
STEPS

10
ANNOTATED FIGURES

1
PYTHON TEMPLATE

Worked example included: customer churn dataset (1,515 rows, 9 columns)

October 2026 | Version 1.0

CONTENTS

Contents

What EDA Is, and the Workflow . . . 3
Step 1. Frame the Problem . . . 4
Step 2. Take a First Look . . . 5
Step 3. Assess Data Quality . . . 6
Step 4. Study Numeric Variables One at a Time . . . 7
Step 5. Detect and Judge Outliers . . . 9
Step 6. Study Categorical Variables . . . 10
Step 7. Explore Relationships Between Two Variables . . . 11
Step 8. Measure Correlation, and Distrust It . . . 12
Step 9. Look for Patterns Over Time . . . 13
Step 10. Compare Segments Against the Target . . . 14
Step 11. Summarise and Document . . . 15
Chart Chooser and Common Pitfalls . . . 16
Reusable Python Template . . . 17

The Example Dataset

Every figure uses one synthetic dataset: **1,515 customer records** from a subscription business. Each row is one customer; the question is *"Who churns, and why?"* It deliberately contains real-world problems: missing values, duplicates, a skewed income column, and extreme values.

Column	Type	Meaning
customer_id	identifier	Unique customer number (never analyse as a quantity)
signup_date	date	When the customer joined
age, income	numeric	Customer attributes (both have missing values)
area_type, segment	categorical	Urban / Suburban / Rural; Basic / Standard / Premium
tenure_months, monthly_spend	numeric	Months since sign-up; average monthly spend
churned	binary target	1 = left the service, 0 = stayed

OVERVIEW

What EDA Is, and the Workflow

Exploratory Data Analysis (EDA) is the disciplined process of looking at your data before you model it or draw conclusions from it. Its purpose is not to produce attractive charts; it is to answer five practical questions:

- **Structure:** What does one row represent, and what columns do I have?
- **Quality:** What is missing, duplicated, mistyped, or impossible?
- **Distribution:** What does each variable look like on its own?
- **Relationships:** Which variables move together, and how?
- **Story:** What should the reader (or the model) take away?

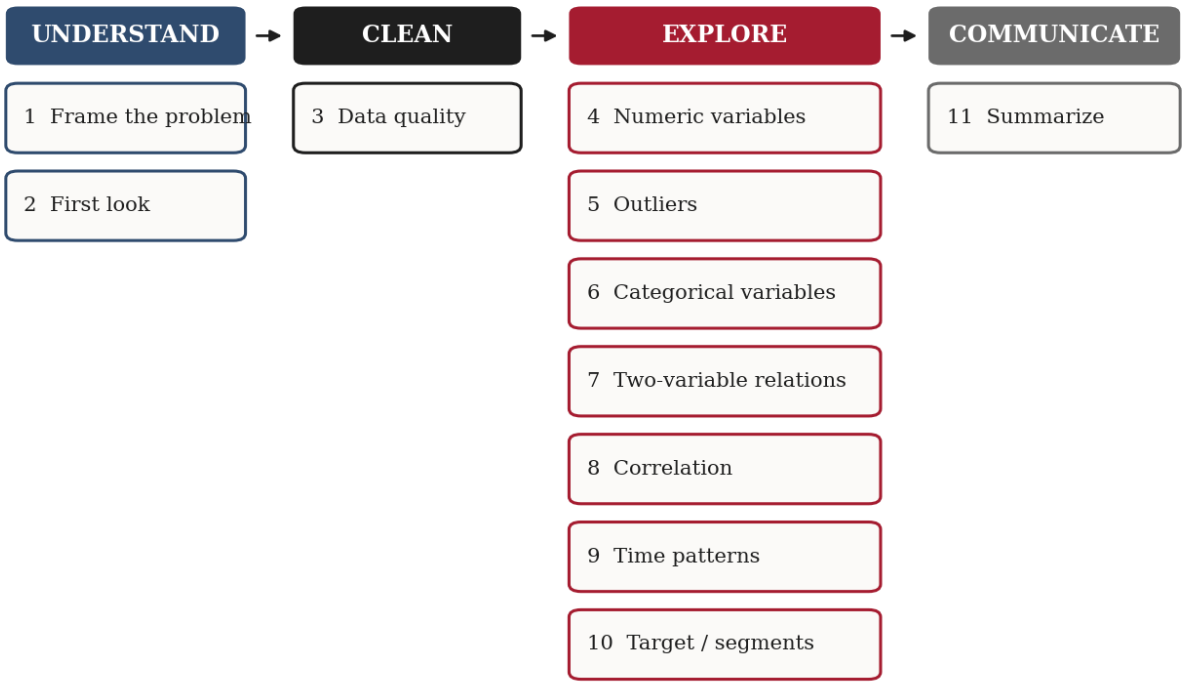


Figure 1. *The 11-step EDA workflow. Steps 4-10 are the exploration core; their order is flexible, but Steps 1-3 should always come first.*

HOW TO USE THIS GUIDE

Read the steps in order the first time. Afterwards, treat each step as a checklist item: copy the code, replace the column names with yours, and answer the "What to look for" questions. Everything is demonstrated on one example dataset so you can see what each chart looks like on real (messy) data.

PHASE 1: UNDERSTAND

Step 1. Frame the Problem

GOAL

Know what you are looking for before you look. EDA without a question turns into aimless chart-making.

Before opening the data, write down the answers to the six prompts below. They take ten minutes and prevent days of wandering. The right-hand column shows how we filled it in for the example dataset.

Prompt	Why it matters	Example answer
Question	Defines which analyses are relevant	Who churns, and what drives it?
Unit of observation	What one row represents; avoids double counting	One customer
Target variable	The outcome you care about (if any)	churned (0/1)
Time column	Enables trend, seasonality, leakage checks	signup_date
Known limitations	How the data was collected; who is missing	Only customers who signed up 2022-2024
Hypotheses (3-5)	Gives each chart a purpose	Low spend and short tenure increase churn

Checklist

- Read the data dictionary (units, definitions, codes such as 999 = unknown).
- Ask how the data was generated: survey, sensor, transaction log, scrape? Each has different biases.
- Note which variables would not be known at prediction time. They cause *data leakage* later.

PHASE 1: UNDERSTAND

Step 2. Take a First Look

GOAL

Learn the shape and structure of the table, and make sure every column has the right data type.

```
import pandas as pd

df = pd.read_csv("data.csv")              # or read_excel / read_parquet

df.shape                                   # (rows, columns)
df.head()                                  # first 5 rows
df.sample(5, random_state=0)               # random rows: avoids ordering bias
df.info()                                  # data types + non-null counts
df.nunique()                               # distinct values per column

# Fix types early (dates stored as text, categories stored as strings ...)
df["signup_date"] = pd.to_datetime(df["signup_date"])
df["segment"] = df["segment"].astype("category")
```

Here is what `df.head()` shows for the example dataset:

id	signup	age	income	area	segment	tenure	spend	churned
1	2022-01-11	56	nan	Rural	Basic	36	850.5	0
2	2022-01-21	42	29,900	NaN	Basic	35	487.8	0
3	2022-01-23	NaN	22,500	Suburban	Standard	35	571.2	1
4	2022-02-01	47	35,300	Urban	Basic	35	865.0	0
5	2022-02-04	24	73,500	Suburban	Basic	35	757.9	0

What to look for

- **Size:** Is the number of rows what you expected? (A join gone wrong often shows up here.)
- **Types:** Dates stored as text, numbers stored as strings ("1,200"), or IDs stored as numbers.
- **Identifiers:** Is `customer_id` unique? If not, what does a repeat mean?
- **Plausibility:** Do the first and random rows look like real records?

PHASE 2: CLEAN

Step 3. Assess Data Quality

GOAL

Find and document everything that could silently distort later results: missing values, duplicates, and inconsistent labels.

```
# 1. Missing values: % per column
miss = df.isna().mean().mul(100).round(1).sort_values(ascending=False)
print(miss[miss > 0])

# 2. Duplicates
print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates()

# 3. Inconsistent text labels ("urban", "Urban ", "URBAN")
df["area_type"] = df["area_type"].str.strip().str.title()
print(df["area_type"].value_counts(dropna=False))

# 4. Simple imputation: decide per column AFTER studying the pattern
df["age_missing"] = df["age"].isna()              # keep a flag: missingness can be a signal
df["age"] = df["age"].fillna(df["age"].median())
```

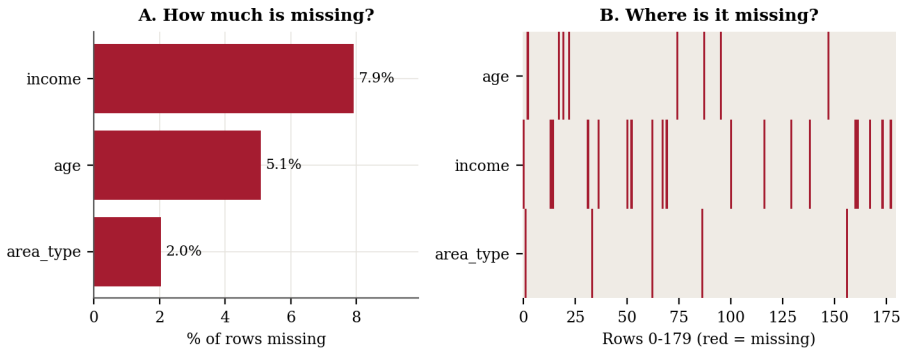


Figure 2. *Missing values in the example data. Panel A: income (7.9%), age (5.1%) and area_type (2.0%) have gaps. Panel B: the gaps are scattered, with no obvious block pattern. We also found* ***15 exact duplicate rows****.*

Decision guide for missing values

Situation	Typical action
Under ~5% missing, scattered randomly	Impute (median for skewed numerics, mode for categories) or drop the rows
5-40% missing	Impute *and* add an "is_missing" flag; check whether missingness relates to the target
Over ~40% missing	Consider dropping the column, unless missingness itself is informative
Missing in blocks (Panel B shows stripes)	Investigate the cause: a system change, a survey skip pattern, a join failure

Whatever you decide, **write it down**. Every cleaning choice is an assumption the reader of your analysis needs to know about.

PHASE 3: EXPLORE

Step 4. Study Numeric Variables One at a Time

GOAL

Understand the centre, spread, and shape of every numeric variable before comparing anything.

```
import numpy as np
import matplotlib.pyplot as plt

num_cols = ["age", "income", "tenure_months", "monthly_spend"]
df[num_cols].describe().T                  # count, mean, std, min, quartiles, max
df[num_cols].skew().round(2)               # > +1 : strong right skew

fig, ax = plt.subplots(1, 2, figsize=(9, 3))
ax[0].hist(df["income"].dropna(), bins=40)                 # raw scale
ax[1].hist(np.log10(df["income"].dropna()), bins=40)       # log scale for skewed data
plt.show()
```

variable	count	mean	std	min	25%	50%	75%	max
age	1,425	38.3	10.8	18.0	31.0	38.0	46.0	74.0
income	1,380	48,723.2	39,199.8	7,400.0	28,300.0	39,200.0	59,300.0	726,783.4
tenure_months	1,500	13.4	9.2	0.0	5.0	12.0	20.0	36.0
monthly_spend	1,500	826.3	384.8	39.9	564.5	763.8	1,011.8	3,176.3

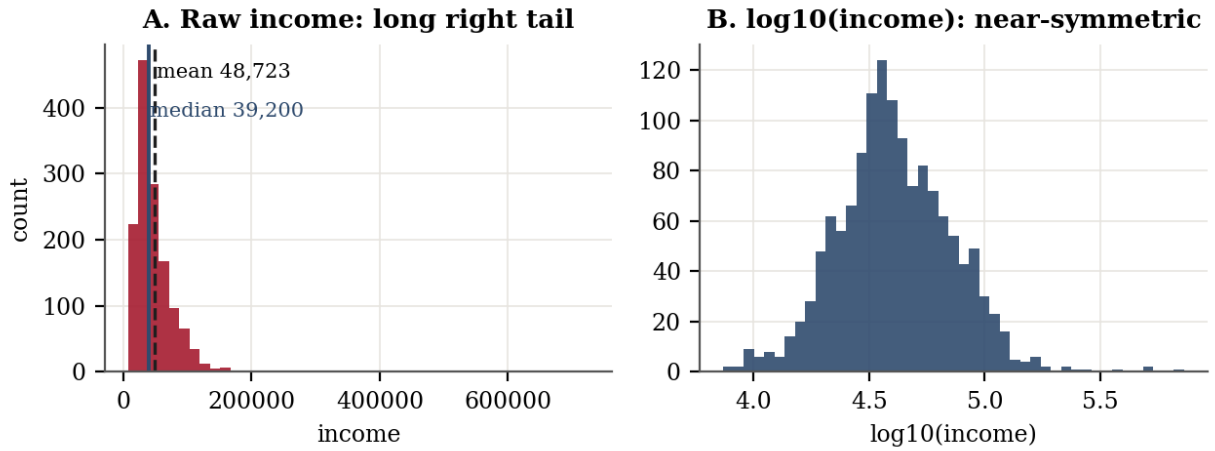


Figure 3. *Income is strongly right-skewed (skew = 7.2): the mean (48,723) sits far above the median (39,200). A log transform makes the shape close to symmetric.*

Reading the shape

What you see	What it usually means	What to do
Symmetric bell	Well-behaved variable	Mean and standard deviation are fine
Long right tail (mean > median)	Money, counts, durations	Report the median; try a log transform
Two peaks (bimodal)	Two groups mixed together	Split by a categorical variable
Spike at one value (0, 999, -1)	Placeholder or default for "unknown"	Treat as missing

Taming skewed variables

Technique	Use when	Code
Log transform	Positive values, long right tail (income, prices)	np.log10(x) or np.log1p(x)
Square-root transform	Counts, mild right skew	np.sqrt(x)
Winsorize (cap)	A few extreme values dominate	x.clip(x.quantile(.01), x.quantile(.99))
Use the median	Reporting a typical value	x.median()

Remember: transforms change how you *see* the data. Keep the original column and record which transform you applied.

PHASE 3: EXPLORE

Step 5. Detect and Judge Outliers

GOAL

Find extreme values, then decide whether each is an error, a legitimate rare case, or a different population.

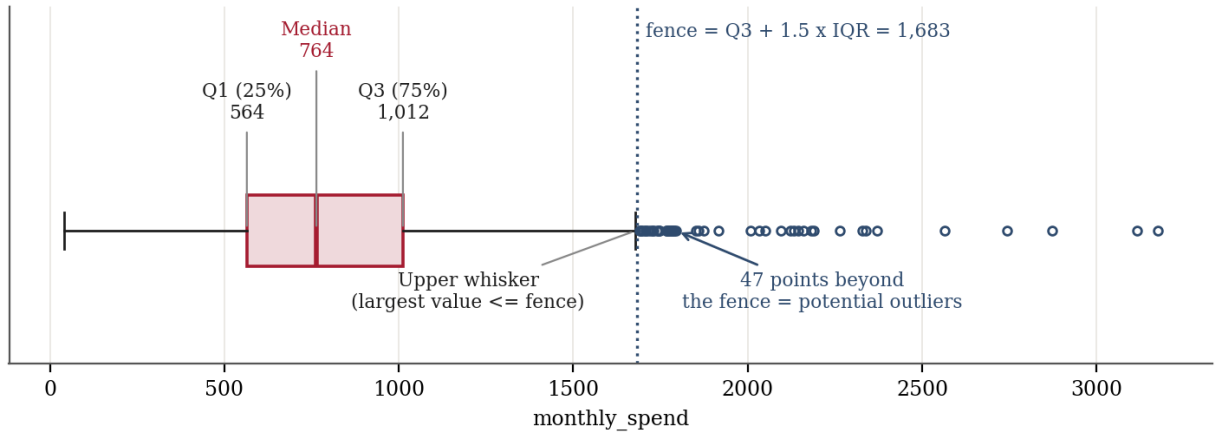


Figure 4. *Anatomy of a boxplot on monthly_spend. The box spans the middle 50% of values (Q1 to Q3). Points beyond 1.5 x IQR from the box are flagged as potential outliers.*

```
def iqr_outliers(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - k * iqr, q3 + k * iqr
    return s[(s < low) | (s > high)], low, high

out, low, high = iqr_outliers(df["monthly_spend"].dropna())
print(f"{len(out)} values outside [{low:.0f}, {high:.0f}]")

# Look at the actual rows before deciding anything
df.loc[out.index].sort_values("monthly_spend", ascending=False).head()
```

Variable	Values flagged by the 1.5 x IQR rule
age	5
income	61
tenure_months	0
monthly_spend	47

THE KEY RULE

An outlier is a **question**, not a verdict. Never delete a value just because it is inconvenient. Investigate it: is it a typo (fix or remove), a real but extreme case (keep; use medians, log scales, or robust methods), or a different kind of customer (analyse separately)?

PHASE 3: EXPLORE

Step 6. Study Categorical Variables

GOAL

Know how many categories there are, how balanced they are, and whether labels are clean.

```
cat_cols = ["area_type", "segment"]
for c in cat_cols:
    print(df[c].value_counts(normalize=True, dropna=False).mul(100).round(1), "\n")

# Group rare categories (< 2%) into "Other"
freq = df["segment"].value_counts(normalize=True)
rare = freq[freq < 0.02].index
df["segment_grouped"] = df["segment"].where(~df["segment"].isin(rare), "Other")
```

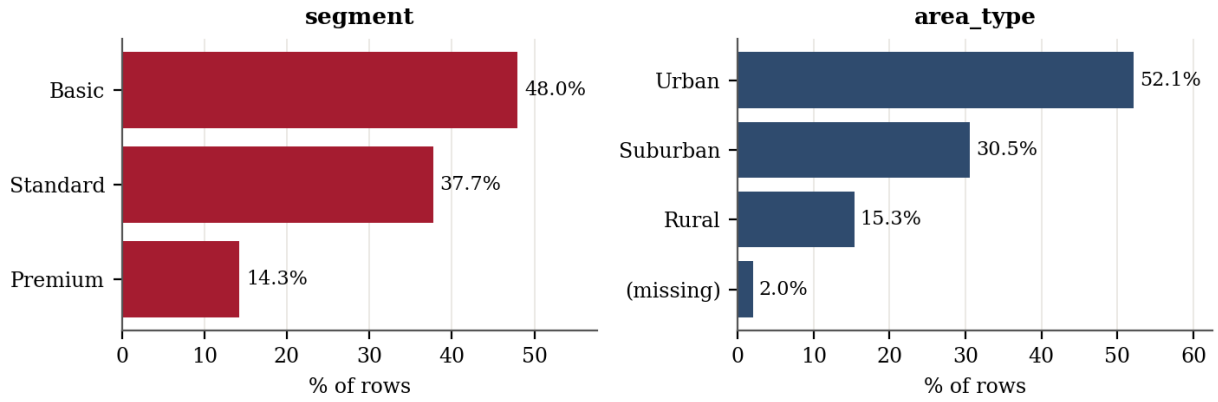


Figure 5. *Share of rows in each category. Segment is imbalanced (Basic is half of all customers); area_type shows its 2% missing values as a separate bar.*

What to look for

- **Cardinality:** A column with hundreds of distinct values (e.g. city names, product codes) needs grouping or a different treatment than a column with three.
- **Imbalance:** Rare categories (under 1-2%) give unstable statistics. Group them into "Other" or report with caution.
- **Label hygiene:** "Urban", "urban " and "URBAN" are three categories to a computer.
- **Missing as information:** Sometimes "(missing)" deserves to stay as its own category.

PHASE 3: EXPLORE

Step 7. Explore Relationships Between Two Variables

GOAL

See how pairs of variables relate. The right chart depends on the types of the two variables.

Variable types	Best chart	Numeric summary
Numeric vs numeric	Scatter plot (add a trend line)	Correlation coefficient
Numeric vs categorical	Box plot or violin plot per group	Mean / median per group
Categorical vs categorical	Stacked or grouped bar chart	Cross-tabulation (percentages)

```
import seaborn as sns

sns.scatterplot(data=df, x="income", y="monthly_spend", hue="segment", alpha=0.5)   # num vs num
plt.show()
sns.violinplot(data=df, x="segment", y="monthly_spend")                             # num vs cat
plt.show()
pd.crosstab(df["segment"], df["area_type"], normalize="index").mul(100).round(1)    # cat vs cat
```

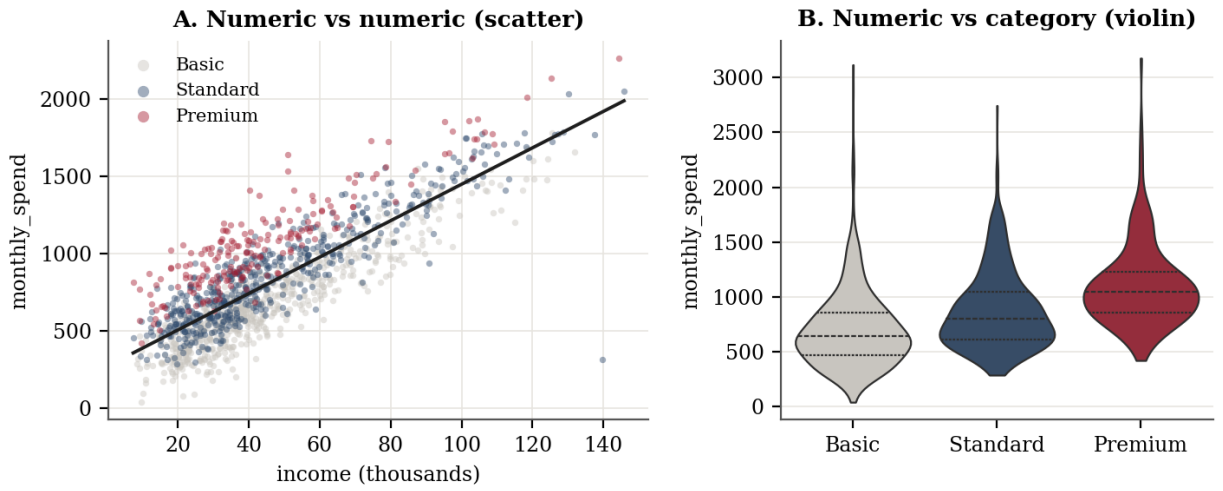


Figure 6. *Panel A: spend rises with income (Spearman rho = 0.79); extreme incomes above 150k are cropped for readability. Panel B: Premium customers spend more on average, but the distributions overlap heavily.*

Tip: When points overlap heavily, use transparency (`alpha`), hexbin plots, or a sample. Always colour by a categorical variable once to check whether subgroups behave differently.

PHASE 3: EXPLORE

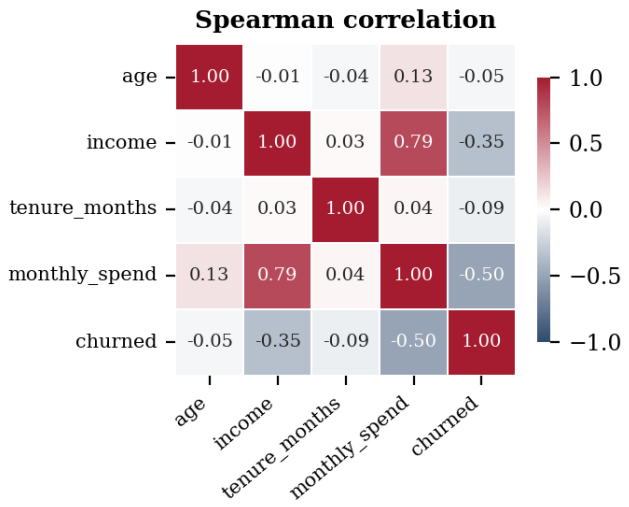
Step 8. Measure Correlation, and Distrust It

GOAL

Summarise all pairwise relationships at once, find redundant variables, and spot candidate predictors.

```
corr = df[num_cols + ["churned"]].corr(method="spearman")   # robust to outliers
sns.heatmap(corr, annot=True, fmt=".2f", cmap="vlag", center=0, vmin=-1, vmax=1)
plt.show()

# Rank every variable by its relationship with one column
corr["monthly_spend"].drop("monthly_spend").sort_values(ascending=False)
```



Reading |r| (absolute value)

0.0 - 0.3 Weak. Often negligible in practice.

0.3 - 0.7 Moderate. Worth investigating.

0.7 - 1.0 Strong. Between two *predictors* above ~0.9, one is redundant (multicollinearity).

Here: income and spend move together; spend and churn move in opposite directions.

Figure 7. *Spearman correlation matrix. Red = positive, blue = negative.*

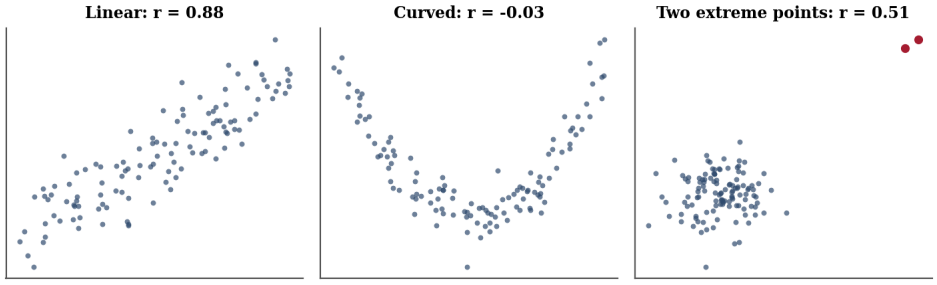


Figure 8. *Three datasets, very different structure. A correlation coefficient measures only linear association and is easily distorted by a few extreme points. Always plot before trusting r.*

Pearson measures linear relationships; **Spearman** measures rank (monotonic) relationships and is more robust. Remember: correlation is not causation.

PHASE 3: EXPLORE

Step 9. Look for Patterns Over Time

GOAL

If the data has a date column, check trend, seasonality, gaps, and sudden changes. Skip this step if there is no time dimension.

```
df["signup_date"] = pd.to_datetime(df["signup_date"])
monthly = df.set_index("signup_date").resample("MS").size()   # count per month

fig, ax = plt.subplots(figsize=(8, 3))
monthly.plot(ax=ax, label="monthly count")
monthly.rolling(3).mean().plot(ax=ax, label="3-month rolling mean")
ax.legend()
plt.show()
```

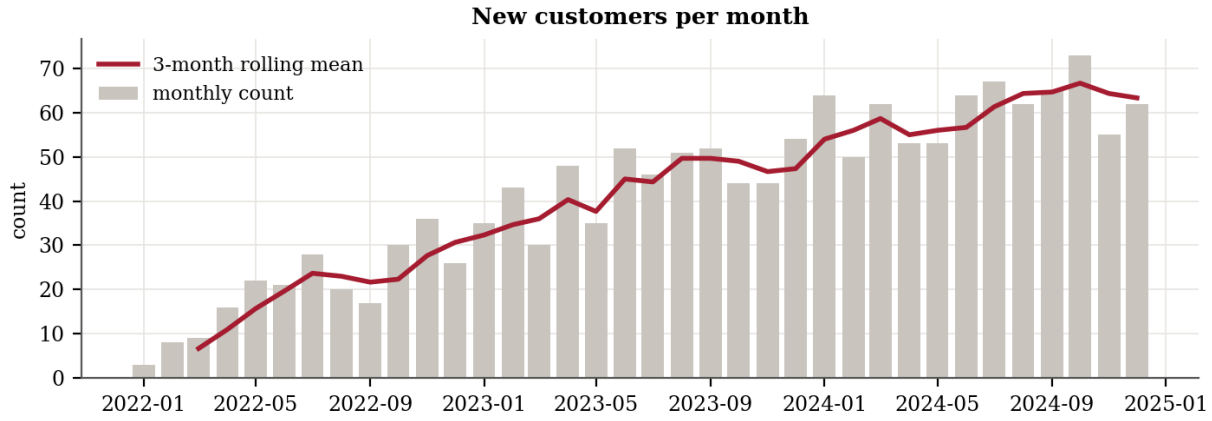


Figure 9. *New customers per month. Sign-ups grew each year (236 in 2022, 534 in 2023, 730 in 2024). The rolling mean smooths month-to-month noise.*

What to look for

- **Trend:** a steady rise or fall over the whole period.
- **Seasonality:** repeating weekly, monthly, or yearly cycles.
- **Gaps and breaks:** missing periods or sudden level shifts often point to a system change, not real behaviour.
- **Leakage:** when you later build a model, split train and test by time, not randomly.

PHASE 3: EXPLORE

Step 10. Compare Segments Against the Target

GOAL

Connect everything back to the question from Step 1: how does the outcome differ across groups?

```
overall = df["churned"].mean()
by_seg = df.groupby("segment")["churned"].agg(rate="mean", n="size")
print(f"Overall churn: {overall:.1%}")
print(by_seg.sort_values("rate", ascending=False))

df["tenure_band"] = pd.cut(df["tenure_months"], [-1, 6, 12, 24, 100],
                           labels=["0-6", "7-12", "13-24", "25+"])
fig, ax = plt.subplots(figsize=(5, 3))
df.groupby("tenure_band", observed=True)["churned"].mean().plot.bar(ax=ax)
plt.show()
```

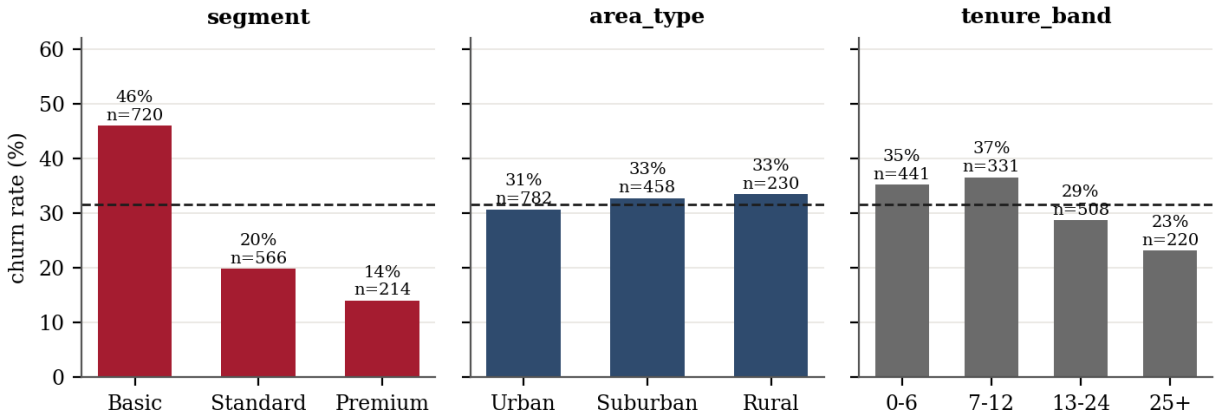


Figure 10. Churn rate by group. The dashed line is the overall rate (31.5%). Segment matters a lot (Basic 46% vs Premium 14%); the area_type differences are small (31% to 34%).

Good habits

- **Always show a baseline** (the overall rate) so differences are meaningful.
- **Show the group size (n).** A 60% churn rate on 8 customers is noise.
- **Separate big from small effects.** Here segment is a strong signal; area_type is a weak one and should not be over-interpreted.
- **Beware Simpson's paradox:** a pattern that appears in the combined data can reverse inside every subgroup. Check key relationships within segments.

PHASE 4: COMMUNICATE

Step 11. Summarise and Document

GOAL

Turn exploration into a short, honest record of what you found, what you changed, and what to do next.

Use this five-part template. The example on the right is filled in from the worked dataset.

Section	Example
1. Data and quality	1,515 rows, 9 columns. Removed 15 duplicate rows. Missing: income 7.9%, age 5.1%, area_type 2.0%.
2. Key patterns (with numbers)	Overall churn is 31.5%. Basic-segment churn (46%) is more than three times Premium (14%). Churn is highest in the first year of tenure (35-37%) and falls to 23% after two years.
3. Hypotheses checked	Supported: lower spend relates to higher churn (rho = -0.50). Weak: area type (differences of about 3 points).
4. Caveats	Income is heavily skewed (skew 7.2); extreme values were kept but a log scale was used. Correlation does not show causation.
5. Next steps	Impute age and income; log-transform income; build a baseline churn model; investigate why first-year customers leave.

Final checklist

[ ]	Problem, unit of observation, and target written down
[ ]	Types fixed; duplicates handled; missing values quantified and decided
[ ]	Every numeric variable plotted (histogram + boxplot); skew noted
[ ]	Outliers investigated, not blindly removed
[ ]	Every categorical variable counted; rare and messy labels handled
[ ]	Key pairs plotted; correlations checked with scatter plots
[ ]	Time patterns checked (if applicable)
[ ]	Target compared across groups, with baselines and group sizes
[ ]	Findings, caveats, and next steps documented

APPENDIX A

Chart Chooser and Common Pitfalls

Question you are asking	Use	Avoid
How is one numeric variable distributed?	Histogram, box plot, density plot	Pie charts, bar of the mean only
How do categories compare?	Sorted horizontal bar chart	3D charts, unsorted bars
How are two numeric variables related?	Scatter plot with trend line	Reporting r without a plot
How does a number differ by group?	Box or violin plot per group	Bars of the mean with no spread
How does it change over time?	Line chart, rolling mean	Many overlapping lines
How do many numeric variables relate?	Correlation heatmap, pair plot	Heatmaps with no annotation
What share is each part of a whole?	Stacked bar, 100% bar	Pie charts with many slices

Ten common pitfalls

- Skipping Step 1 and exploring without a question.
- Treating an ID column (customer_id) as a numeric variable. It will show up in summaries and correlations.
- Dropping rows with missing values without checking whether they are a special group.
- Using the mean on skewed data instead of the median.
- Deleting outliers without investigating them.
- Trusting a correlation coefficient without plotting the data.
- Reading causation into correlation.
- Ignoring group sizes when comparing rates.
- Making charts without titles, units, or axis labels.
- Not recording the cleaning decisions you made.

APPENDIX B

Reusable Python Template

The complete script is provided as `eda_template.py`. It runs Steps 2-10 automatically on any CSV file and saves a figure for every variable. Run it first for a quick overview, then use the individual steps above to dig deeper.

```
# Command line
python eda_template.py data.csv                    # basic run
python eda_template.py data.csv churned signup_date  # + target + date column

# In a notebook
from eda_template import run_eda
run_eda(df, target="churned", date_col="signup_date")
```

Tips for adapting it

- Drop identifier columns before running: `df = df.drop(columns=["customer_id"])`.
- Convert date columns first: `pd.to_datetime(df["signup_date"])`.
- For a target with many numeric values (regression), the template draws scatter plots with trend lines; for a binary target it draws box plots and group averages.
- Use the template as a starting point, then follow your own questions from Step 1.

Template source

```
"""Reusable EDA template.

Usage:
    python eda_template.py data.csv                    # basic run
    python eda_template.py data.csv churned signup     # with target + date column
"""
import os
import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

sns.set_theme(style="whitegrid", context="notebook")
RED = "#A51C30"


def split_columns(df, max_cat=20):
    """Return lists of numeric and categorical column names."""
    num = df.select_dtypes("number").columns.tolist()
    cat = [c for c in df.columns
           if c not in num
           and not pd.api.types.is_datetime64_any_dtype(df[c])
           and df[c].nunique() <= max_cat]
    return num, cat


def overview(df):
    """Step 2-3: structure, duplicates, missing values, summary stats."""
    print("Shape:", df.shape)
    print("\nColumn types:\n", df.dtypes.value_counts())
    print("\nDuplicate rows:", df.duplicated().sum())
    miss = df.isna().mean().mul(100).round(1).sort_values(ascending=False)
    print("\nMissing % (top 10):\n", miss.head(10))
    print("\nNumeric summary:\n", df.describe().T.round(2))


def save(fig, path):
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    plt.close(fig)


def plot_missing(df, out):
    miss = df.isna().mean().mul(100).sort_values()
    miss = miss[miss > 0]
    if miss.empty:
        return
    fig, ax = plt.subplots(figsize=(6, 0.4 * len(miss) + 1.5))
    miss.plot.barh(ax=ax, color=RED)
    ax.set_xlabel("% missing")
    save(fig, f"{out}/01_missing.png")


def plot_numeric(df, num, out):
    """Step 4-5: histogram + boxplot for every numeric column."""
    for c in num:
        fig, ax = plt.subplots(1, 2, figsize=(9, 3))
        sns.histplot(df[c].dropna(), kde=True, ax=ax[0], color=RED)
        sns.boxplot(x=df[c], ax=ax[1], color="#C3C3C3")
        ax[0].set_title(f"{c}  (skew = {df[c].skew():.2f})")
        save(fig, f"{out}/num_{c}.png")


def plot_categorical(df, cat, out):
    """Step 6: frequency of each category."""
    for c in cat:
        order = df[c].value_counts().index
        fig, ax = plt.subplots(figsize=(6, 3))
```

```
        sns.countplot(data=df, y=c, order=order, ax=ax, color=RED)
        ax.set_title(f"{c}: category counts")
        save(fig, f"{out}/cat_{c}.png")


def plot_correlation(df, num, out):
    """Step 8: correlation heatmap (Spearman is robust to outliers)."""
    if len(num) < 2:
        return
    corr = df[num].corr(method="spearman")
    fig, ax = plt.subplots(figsize=(0.8 * len(num) + 3, 0.7 * len(num) + 2))
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="vlag", center=0,
                vmin=-1, vmax=1, ax=ax)
    ax.set_title("Spearman correlation")
    save(fig, f"{out}/corr.png")


def plot_target(df, target, num, cat, out):
    """Step 9: how every feature relates to the target."""
    for c in num:
        if c == target:
            continue
        fig, ax = plt.subplots(figsize=(6, 3))
        if df[target].nunique() <= 5:
            sns.boxplot(data=df, x=target, y=c, ax=ax, color="#C3C3C3")
        else:
            sns.regplot(data=df, x=c, y=target, ax=ax, scatter_kws={"alpha": .3},
                        line_kws={"color": RED})
        save(fig, f"{out}/target_{c}.png")
    if df[target].nunique() <= 5:
        for c in cat:
            if c == target:
                continue
            fig, ax = plt.subplots(figsize=(6, 3))
            rate = df.groupby(c)[target].mean().sort_values()
            rate.plot.barh(ax=ax, color=RED)
            ax.set_xlabel(f"mean of {target}")
            save(fig, f"{out}/target_{c}.png")


def plot_time(df, date_col, num, out):
    """Step 10: record count and trend over time."""
    s = pd.to_datetime(df[date_col], errors="coerce").dropna()
    fig, ax = plt.subplots(figsize=(8, 3))
    s.dt.to_period("M").value_counts().sort_index().plot(ax=ax, color=RED)
    ax.set_title(f"Records per month ({date_col})")
    save(fig, f"{out}/time_counts.png")


def run_eda(df, target=None, date_col=None, out="eda_output"):
    os.makedirs(out, exist_ok=True)
    overview(df)
    num, cat = split_columns(df)
    plot_missing(df, out)
    plot_numeric(df, num, out)
    plot_categorical(df, cat, out)
    plot_correlation(df, num, out)
    if target:
        plot_target(df, target, num, cat, out)
    if date_col:
        plot_time(df, date_col, num, out)
    print(f"\nDone. Figures saved in ./{out}/")


if __name__ == "__main__":
    data = pd.read_csv(sys.argv[1])
    tgt = sys.argv[2] if len(sys.argv) > 2 else None
    dcol = sys.argv[3] if len(sys.argv) > 3 else None
    run_eda(data, tgt, dcol)
```